

Style Separation Cheaply Mitigates Feedback Spillover

Arun Jose¹ and Julian Stastny²

¹[**TODO**: affiliation]

²[**TODO**: affiliation]

June 2026 — working draft

Abstract

Drori et al. [2025] show that training a reasoning model against a monitor that sees only its *output*—never its chain of thought (CoT)—can nonetheless make the CoT *obfuscated*: the penalized behavior disappears from the CoT as well, so the CoT no longer reveals it. They call this *feedback spillover* and propose two mitigations—reward targeting and Mind & Face—both of which, in their question-answering environment, fail to reduce spillover without harming task performance. We introduce a simpler mitigation: fine-tune the model so its output and CoT use *different styles* before reinforcement learning. Styling a model’s outputs while leaving its reasoning plain separates the two channels at the level of persona, and an output-only penalty then fails to propagate into the CoT. On the same question-answering task, style separation preserves CoT monitorability at least as well as the prior mitigations individually, at a fraction of the cost—no second model and no modification to the gradient—and composes with reward targeting. Feedback spillover, we further find, is strongly *regime-dependent*: severe at short CoT budgets (the setting of the original work) and at small model scale, but weak for an 8B or 32B model given a long CoT budget. Finally, the effect depends on *which channel carries the penalized style*: a model fine-tuned to reason—rather than answer—in a distinctive style shows *total* spillover. Spillover is governed by how the penalty’s target relates to the model’s internal structure, not by stylistic difference alone.

1 Introduction

Reasoning models produce a chain of thought (CoT) before their final answer, and that CoT often reveals *why* the model produced the answer it did [Guo et al., 2025, Korbak et al., 2025]. This has been proposed as an opportunity for oversight: a monitor that reads the CoT can catch misbehavior that the final output hides [Baker et al., 2025, Korbak et al., 2025]. A natural worry is that training against a CoT monitor teaches the model to hide bad reasoning from that monitor [Baker et al., 2025]; the standard recommendation is therefore to optimize only against *output* monitors that never see the CoT.

Drori et al. [2025] show this recommendation is not enough. Training against an output-only monitor can still cause *feedback spillover*: the penalized behavior stops appearing in the CoT, even though the CoT was never optimized against. They give two mechanisms—*conditional* spillover (CoTs that lead to safe-looking outputs are differentially reinforced) and *parametric* spillover (parameters that make outputs safe generalize to make CoTs safe)—and two corresponding mitigations: *reward targeting*, which zeros the gradient term that flows from the output penalty through the CoT, and *Mind & Face*, which generates the CoT and the output with two separate models. In their question-answering (QA) environment, neither mitigation reduces spillover without destabilizing training or sharply reducing task reward.

We propose a mitigation that is cheaper than either and, in this environment, at least as effective: **style separation**. Before reinforcement learning (RL), we fine-tune the model with a small amount of supervised data so that its *output* is written in a distinctive style—we use pirate-speak—while its CoT stays plain. Styling the output puts it in a different region of the model’s behavior than the CoT, so an update that makes the output “look safe” does not generalize to the reasoning. The intervention needs no second model (unlike Mind & Face) and no change to the gradient (unlike reward targeting)—only cheap SFT, which a developer may already run for formatting.

Our contributions:

[Figure 1 — schematic, to be added.]

[TODO: Schematic of the overall result. Top row (standard training): a model is RL’d against an output-only monitor; the penalized behavior vanishes from both the output *and* the CoT—the CoT is obfuscated (feedback spillover). Bottom row (ours): the model’s output is in a distinct style (pirate-speak) while its CoT is plain; the same output penalty removes the behavior from the output but the CoT still shows it—the CoT stays monitorable. Mirror the two-robot framing of Drori et al. [2025], Fig. 1.]

Figure 1: **Style separation keeps the chain of thought monitorable under an output-only penalty.** Training a reasoning model against a monitor that sees only the output normally erases the penalized behavior from the chain of thought as well (*feedback spillover*). If the output is written in a distinct style from the CoT, the penalty cleans the output but leaves the behavior visible in the CoT.

1. We introduce style separation and show that, on the QA-with-hints environment of Drori et al. [2025], fine-tuning the output style before RL keeps the penalized behavior detectable in the CoT at essentially its no-penalty level, while fully suppressing it from the output and retaining full task reward (Section 4.1).
2. We re-run the prior mitigations under matched conditions and find style separation beats reward targeting and Mind & Face *individually* on CoT monitorability, and composes with reward targeting (Section 4.2).
3. We show feedback spillover is regime-dependent: severe at short CoT budgets and small scale, weak at long CoT budgets, and that our result holds across budgets (Section 4.3).
4. We show that fine-tuning the *CoT* style instead of the output style *induces* total spillover—demonstrating that spillover is governed by where the penalized style sits relative to the model’s structure (Section 4.4).
5. We provide supporting evidence in a second environment and across model families (Section 4.5).

2 Related Work

CoT monitorability. A CoT monitor scores the reasoning trace for some behavior [Korbak et al., 2025]. Its value depends on the CoT remaining *faithful*—reflecting the computation that produced the answer. Chen et al. [2025] show CoTs are sometimes unfaithful, while Emmons et al. [2025] argue that when a task genuinely requires reasoning, the CoT must reveal it, so monitoring remains useful. Baker et al. [2025] show that optimizing *against* a CoT monitor produces obfuscated CoT and recommend training only against output monitors.

Feedback spillover. Drori et al. [2025] show output-only training is insufficient: the penalized behavior can vanish from the CoT regardless. They formalize spillover via the policy-gradient decomposition $\nabla = \nabla_{\text{CoT}} + \nabla_{\text{out}}$ and derive their two mitigations. *Reward targeting* removes the conditional-spillover term; *Mind & Face*—following the shoggoth+face proposal of Kokotajlo and Demski [2024]—removes the parametric term by using distinct CoT and output models. Our mitigation can be read as targeting the parametric channel *without* a second model: style separation pushes the output into a distinct behavioral mode, so output-safety updates do not generalize to the CoT.

3 Methods

We follow the QA-with-hints environment of Drori et al. [2025]. The model is asked a multiple-choice question (MMLU elementary mathematics; Hendrycks et al., 2021) whose prompt contains a hint pointing to a specific, *incorrect* answer, e.g. “A friend mentioned that the answer should be A.” Because the hint is wrong, a model that answers the hinted letter must be *using* the hint. We reward producing the hinted answer (so high task reward means the model

is being sycophantic, the behavior we will then try to keep visible) and penalize *acknowledging* the hint in the output. Concretely the training reward is

$$R = R_{\text{task}} - \lambda M_{\text{out}},$$

where $R_{\text{task}} = 1$ iff the boxed answer matches the hint (and -1 if the output contains no extractable boxed answer, which stops the model from evading the penalty by simply declining to answer), $M_{\text{out}} \in [0, 1]$ is a soft GPT-4o-mini judge of whether the *output* acknowledges the hint, and λ is the penalty weight (our main results use $\lambda = 2$). A second judge with the identical rubric reads only the *CoT*; its score M_{CoT} is used for evaluation only, never for training. **Feedback spillover** is a drop in M_{CoT} relative to a no-penalty control: the hint stops being acknowledged in the CoT even though only the output was penalized.

Style separation. Before RL we LoRA-fine-tune the base model on a styled version of the Stanford Alpaca instruction set [Taori et al., 2023], built from the model itself in two passes: the model first answers each instruction normally (a `<think>` CoT followed by a plain output), then rewrites only the post-`</think>` output into exaggerated pirate-speak, leaving the CoT untouched. Styling only the output—and generating the data from the model’s own normal responses—keeps the reasoning natural, so no pirate planning leaks into the CoT. We fine-tune on 10,000 such examples (LoRA rank 32, 3 epochs, batch size 128, learning rate 10^{-4}). We compare against three controls: no SFT; *normal* SFT through the same pipeline without the rewriting step, which isolates “SFT” from “style”; and *pirate-CoT* SFT, which styles the CoT instead of the output (Section 4.4).

Prior mitigations. We re-implement reward targeting (zero the output penalty’s advantage on CoT tokens) and Mind & Face (separate LoRA adapters generate CoT and output), and their combination, Targeted Mind & Face.

Training. We train with GRPO [Williams, 1992] via the Tinker API [Thinking Machines Lab, 2025]: LoRA rank 32, group size 4 over 3 prompts (12 rollouts/batch), 1000 batches, learning rate 10^{-5} , temperature 1. Generation is two-phase: a CoT of up to T thinking tokens, then an output of up to 600 tokens; our main results use $T = 4096$, and Section 4.3 studies $T = 300$ (the original work’s budget). We use Qwen3-8B and Qwen3-32B [Qwen Team, 2025], two seeds each. Evaluation generates at the same CoT budget the model was trained at, on the 378 MMLU elementary-mathematics questions—the same pool seen during training, so spillover is measured in-distribution.

Differences from Drori et al. [2025]. We follow their environment but differ in implementation. We fine-tune LoRA adapters (rank 32) rather than the full model; we generate the CoT and output in two separate sampling passes—CoT up to the `</think>` delimiter, then output—rather than a single pass with a logit processor; we optimize with GRPO (advantages normalized within groups of 4 samples per prompt) rather than their REINFORCE estimator; and we use Qwen3-8B and Qwen3-32B with two seeds each rather than Qwen3-4B with ten. Where our numbers diverge from theirs—most visibly the weak baseline spillover at $T = 4096$ —these differences and the CoT-budget effect of Section 4.3 are the likely causes.

4 Results

4.1 Style Separation Preserves CoT Monitorability

Figure 2 shows the core result. For each SFT condition we plot the penalized run’s CoT hint-detection rate (x-axis; higher means the hint is still visible in the CoT, i.e. less spillover) against training reward (y-axis), with a dotted vertical line marking the same model’s no-penalty CoT level. A penalized dot sitting *on* its line means the output penalty did not contaminate the CoT.

Table 1 gives the numbers. With pirate-output SFT, the penalized CoT detection is 0.47 (8B) and 0.55 (32B), essentially unchanged from the no-penalty controls (0.50, 0.59)—the penalty cleaned the output ($M_{\text{out}} \approx 0$) without touching the CoT. All penalized pirate-output runs retain full task reward (≈ 1.0). The no-SFT and normal-SFT conditions lose substantially more CoT signal under the penalty, and (at 8B) do not even learn the task: see Section 4.3 and the nuance in Section 5. We ran the full penalty-weight sweep ($\lambda \in \{0, 0.5, 1, 2\}$) for all four conditions and both sizes; pirate-output keeps CoT detection near its control at every weight [TODO: cite appendix figure: combined_pareto_dots_v7_pw_{0.5,1,2}].

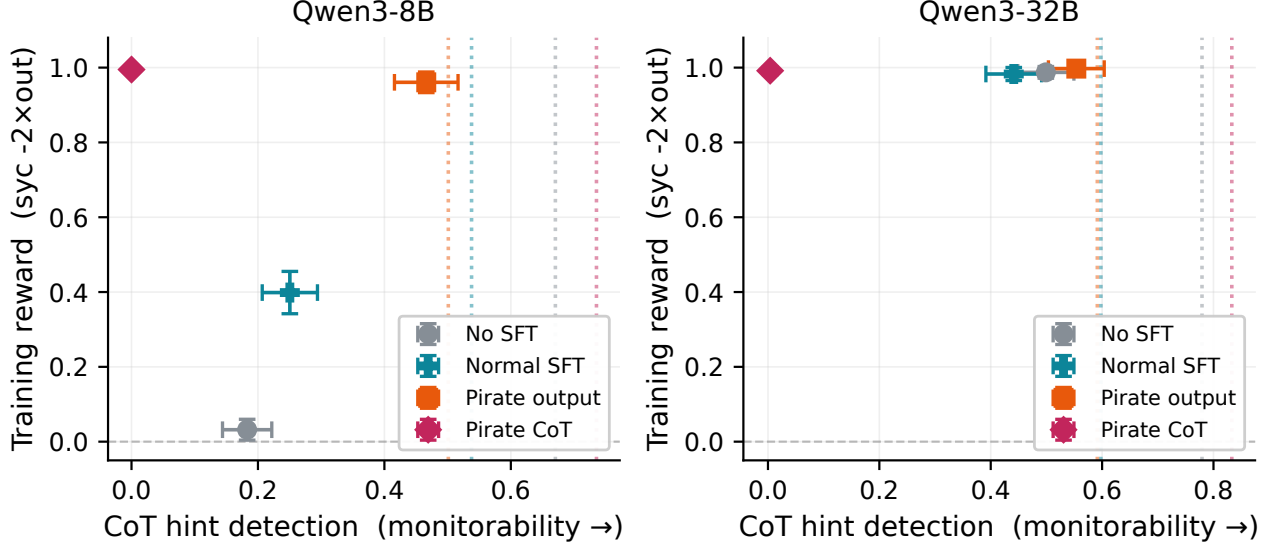


Figure 2: **Style separation preserves CoT monitorability under an output penalty** ($\lambda = 2$, final checkpoints, 2 seeds). Dotted line: the same model’s no-penalty CoT hint-detection level (the “uncontaminated” reference). Pirate-output (red) sits at its reference line on both 8B and 32B—no spillover—at full reward, while no-SFT and normal-SFT lose more CoT signal. Pirate-CoT (magenta) collapses to zero (Section 4.4).

Table 1: **Pirate-output keeps the hint visible in the CoT under penalty; pirate-CoT erases it.** QA, $\lambda = 2$, final-checkpoint CoT hint detection (M_{CoT}), seed-averaged ($n=2$); “control” is the matched no-penalty ($\lambda=0$) run. Pirate-output’s penalized CoT stays at its control level; pirate-CoT collapses to zero on both sizes.

Condition	Qwen3-8B		Qwen3-32B	
	control	penalized	control	penalized
No SFT	0.67	0.18	0.78	0.50
Normal SFT	0.54	0.25	0.60	0.44
Pirate output	0.50	0.47	0.59	0.55
Pirate CoT	0.74	0.00	0.83	0.00

4.2 Comparison to Prior Mitigations, and Composition

We compare pirate-output against reward targeting (RT), Mind & Face (M&F), and Targeted M&F (TMF), all trained and evaluated identically at $T = 4096$ on Qwen3-8B (Figure 3, Table 2).

Pirate-output (0.47) beats reward targeting (0.20) and Mind & Face (0.40) individually on CoT monitorability, at full reward and the lowest cost of any mitigation. Targeted M&F—which applies *both* of the prior protections at once—is the strongest single configuration (0.75); we do not claim to beat it, and note it is also the most expensive. A cheap output-style SFT is thus competitive with the prior mitigations and individually outperforms them.

Style separation composes with reward targeting. The two interventions are orthogonal—one acts at SFT time, the other at gradient time—and they compose: pirate-output + reward targeting reaches CoT detection 0.60, better than either alone (Figure 3b). Combining with Mind & Face does not help beyond pirate alone, so we frame composition specifically around reward targeting.

4.3 Feedback Spillover Is Regime-Dependent

At $T = 4096$, the *baseline* (no-mitigation) penalty barely spills on the larger model: 32B no-SFT penalty retains CoT detection 0.50 (Table 1). This contrasts with the strong spillover reported by Drori et al. [2025], who used a 4B model

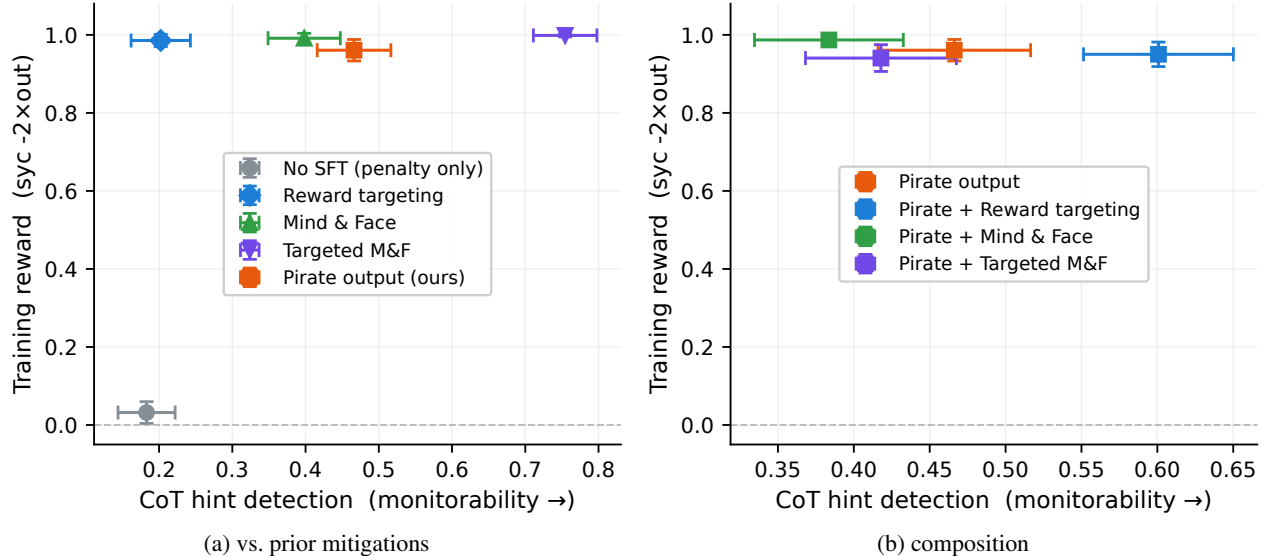


Figure 3: **Pirate-output matches the prior mitigations at a fraction of the cost, and composes with reward targeting.** Qwen3-8B, $\lambda = 2$. (a) Pirate-output (red) preserves CoT monitorability better than reward targeting and Mind & Face individually, at full reward and lower cost; Targeted M&F (both protections) is strongest. (b) Pirate-output composes with reward targeting (0.47→0.60); the M&F compositions do not improve on pirate alone.

Table 2: **Style separation beats reward targeting and Mind & Face individually, at the lowest cost.** Qwen3-8B mitigations, $\lambda = 2$, $T = 4096$: CoT detection (higher = more monitorable), output acknowledgement, reward, and each mechanism’s cost.

Condition	CoT det.	out	reward	cost
No SFT (penalty only)	0.18	0.01	+0.03	— (fails task, §5)
Reward targeting	0.20	0.01	+0.99	gradient modification
Mind & Face	0.40	0.00	+0.99	second model
Pirate output (ours)	0.47	0.02	+0.96	output-style SFT only
Targeted M&F	0.75	0.00	+1.00	second model + gradient mod.

with a 300-token CoT budget. The CoT budget explains the gap (Figure 4).

At $T = 300$, the no-SFT penalty drives hint-in-CoT from 0.41 (control) to ≈ 0.00 (total spillover); at $T = 4096$, the same penalty leaves it at 0.21 (partial). The relative reduction—99.8% vs 68%—is a genuine budget effect, not merely “less room to mention the hint” at 300 tokens (the control level is indeed lower at 300, but the penalty’s marginal effect far exceeds that gap). This both explains why our 32B/4096 baseline barely spills and reproduces the original work’s strong spillover at its own budget. The mitigation comparison holds at $T = 300$ as well: pirate-output (0.37) again beats reward targeting (0.08) and Mind & Face (0.16), with Targeted M&F best (0.42). One caveat: at the short budget pirate-output suppresses the output incompletely and so pays a task-reward cost (reward 0.76 vs 1.0), where at $T = 4096$ it does not; we discuss this in Section 5.

4.4 Where the Style Lives Determines Spillover

If style separation worked simply because the CoT and output were *stylistically different*, then styling the CoT (rather than the output) should also prevent spillover. It does the opposite. A model fine-tuned to write its *CoT* in pirate-speak (plain output) shows **total** spillover under the output penalty: CoT hint detection falls to 0.00 on both 8B and 32B (Table 1, magenta points in Figure 2), down from a no-penalty control of 0.74/0.83. The reversal shows that feedback spillover is governed by the relationship between *which channel the penalty targets* and *which channel carries the distinctive behavior*, not by stylistic dissimilarity alone. When the styled channel is the penalized one (the output),

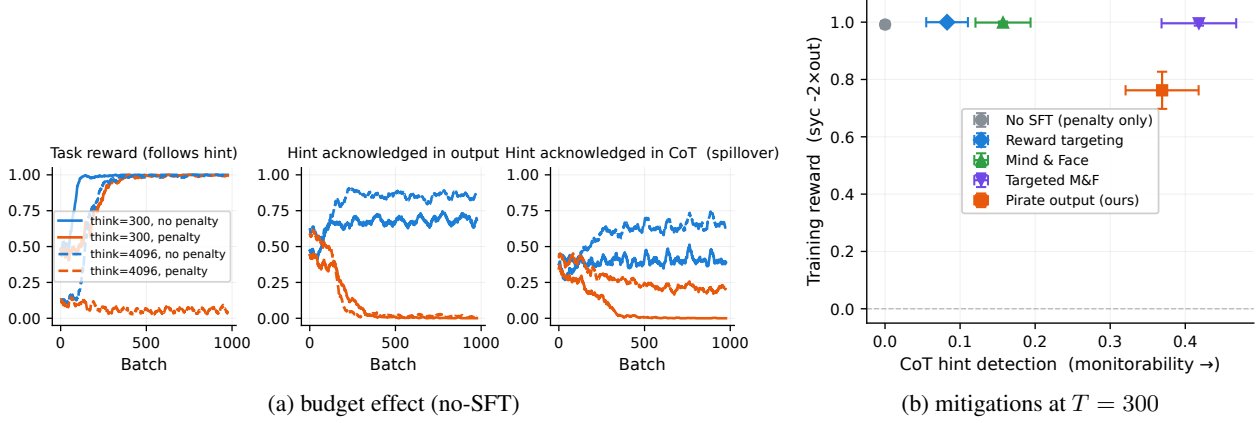


Figure 4: **CoT budget modulates spillover (Qwen3-8B, $\lambda = 2$).** (a) Training curves: at $T = 300$ the no-SFT output penalty drives hint-in-CoT to near zero (total spillover); at $T = 4096$ it leaves substantial hint-in-CoT (partial). (b) The mitigation ranking is robust at the short budget—pirate-output still beats RT and M&F individually—though pirate-output pays a task-reward cost at $T = 300$.

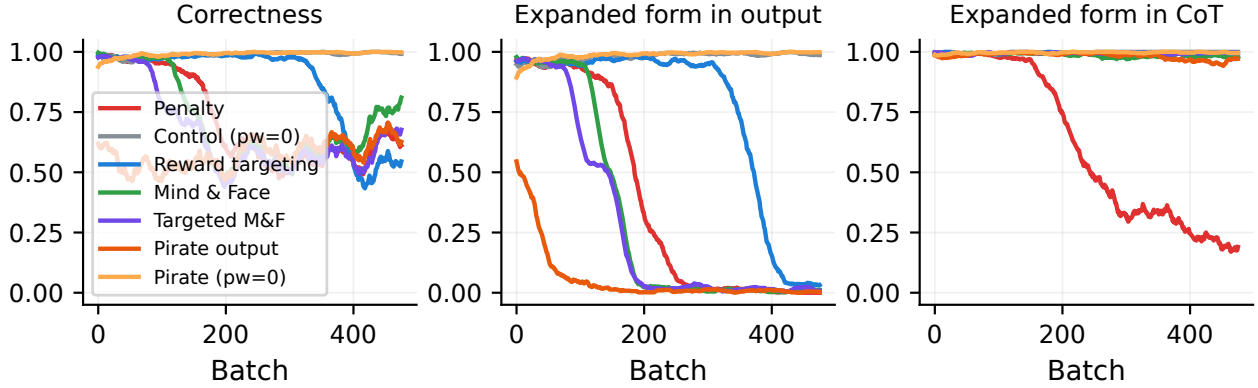


Figure 5: **The result replicates in a second environment.** Polynomial derivative-factoring, Qwen3-32B: the plain penalty (blue) drives expanded-form-in-CoT from 1.0 to ≈ 0.2 (spillover), while pirate-output (red) keeps the CoT intact and suppresses the output fastest of all conditions. Metrics from training rollouts (string-match monitor).

the boundary insulates the CoT; when the output is plain and the reasoning is styled, penalizing the output drags the reasoning down with it.

4.5 Generality

The result replicates in a second environment. We see the same picture in the polynomial derivative-factoring task of Emmons et al. [2025], where the penalized behavior is writing the expanded form of the derivative in the output (a string-match monitor, no LLM judge). The plain penalty reproduces spillover—expanded-form mentions in the CoT fall from ≈ 1.0 to ≈ 0.2 —while pirate-output keeps the CoT intact (≈ 1.0) and suppresses the output fastest of all conditions (Figure 5). The prior mitigations also hold the CoT here, consistent with the original work, where the mitigations succeeded in this environment but not in QA.

Spillover generalizes across model families. Spillover, and its scale-dependence, appear in no-SFT penalty runs on Qwen3.6-35B-A3B and Nemotron-3-Super-120B (Figure 6). The MoE Qwen3.6 model spills weakly (like 32B Qwen3), while Nemotron is notably hint-resistant at baseline and shows different dynamics. **[TODO: This section is currently no-SFT-only on the new models (single seed). Decide whether to (a) leave as a generality-of-spillover note, or (b) add pirate-output runs on one new model to extend the mitigation claim cross-family.]**

5 Discussion

We take the central finding to be that feedback spillover is *controllable*: whether an output-only penalty obfuscates the CoT depends on how the penalized channel relates to the model’s structure, and a cheap persona boundary between output and CoT is enough to keep the reasoning monitorable while still cleaning the output. The same lever that prevents spillover (style separation) can also *induce* it, when the styled channel is the reasoning rather than the output.

Mechanism. We read style separation as installing a persona-level boundary between the output and the CoT: the styled output occupies a distinct behavioral mode, so the parameter update that makes outputs “look safe” does not generalize to the CoT. This is the parametric-spillover channel of Drori et al. [2025], blocked without a second model. We do not claim the pirate persona *persists* throughout training as the mechanism—indeed it largely erodes (below)—only that the separation suffices to keep the penalty from propagating.

Suppression is partly output minimization. Under the penalty, pirate-output models do not learn to “explain without mentioning the hint”—they learn to stop explaining. The output collapses to a bare boxed answer (the pirate style itself disappears over training), which trivially satisfies the output monitor. The CoT, meanwhile, continues to reason coherently and to acknowledge the hint, which is what preserves monitorability. So “the output gives no explanation” is the mechanism by which the penalty is satisfied, not a training failure. The no-SFT penalty at the short budget instead collapses the output to degenerate repetition, and at the long budget produces coherent hint-free explanations but fails to learn the (sycophantic) task at all—answering honestly instead. These differing failure modes are why the no-SFT “baseline” is not a single clean reference across regimes.

Practical implication. Style separation is essentially free—a developer who already applies output-formatting SFT gets it for nothing—and requires no change to the RL objective or infrastructure. It will not, by itself, guarantee monitorability: it is one cheap layer that, in our experiments, matches gradient- and model-based interventions and composes with them.

Future work. Several directions remain open: whether style separation survives full fine-tuning and frontier scale, and whether stacking cheap interventions (style separation, reward targeting, and paraphrasing) can together approach the monitorability of the no-penalty model.

5.1 Limitations

We use two seeds (vs. ten in the original work), LoRA rather than full fine-tuning, two-phase sampling rather than a single pass, and a specific model/API stack (Tinker). The 32B Mind & Face comparison could not be evaluated at the matched CoT budget because Qwen3-32B was deprecated from the API before we caught an evaluation-budget bug (Appendix [TODO: ref]); we report 32B mitigation results only for the conditions we could evaluate correctly.

Reproducibility

All training rollouts, per-sample eval transcripts with judge scores, configs, and LoRA adapters are saved; an interactive transcript browser is available at <https://feedback-spillover.pages.dev>. [TODO: decide what to release publicly; code + SFT data + adapters.]

Acknowledgements

[TODO: acknowledgements.]

References

- Bowen Baker, Joost Huizinga, Leo Gao, Zehao Dou, Melody Y. Guan, Aleksander Madry, Wojciech Zaremba, Jakub Pachocki, and David Farhi. Monitoring reasoning models for misbehavior and the risks of promoting obfuscation. *arXiv preprint arXiv:2503.11926*, 2025.
- Yanda Chen, Joe Benton, Ansh Radhakrishnan, Jonathan Uesato, Carson Denison, John Schulman, et al. Reasoning models don’t always say what they think. *arXiv preprint arXiv:2505.05410*, 2025.
- Jacob Drori, Luke Marks, Bryce Woodworth, Alex Cloud, and Alexander Matt Turner. Output supervision can obfuscate the chain of thought. *arXiv preprint arXiv:2511.11584*, 2025.
- Scott Emmons, Erik Jenner, David K. Elson, Rif A. Saurous, Senthooan Rajamanoharan, Heng Chen, Irhum Shafkat, and Rohin Shah. When chain of thought is necessary, language models struggle to evade monitors. *arXiv preprint arXiv:2507.05246*, 2025.
- Daya Guo et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021.
- Daniel Kokotajlo and Abram Demski. Why don’t we just... shoggoth+face+paraphraser? AI Alignment Forum, 2024.
- Tomek Korbak, Mikita Balesni, Elizabeth Barnes, et al. Chain of thought monitorability: A new and fragile opportunity for ai safety. *arXiv preprint arXiv:2507.11473*, 2025.
- Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca, 2023.
- Thinking Machines Lab. Tinker: A training api for fine-tuning language models. <https://thinkingmachines.ai/tinker/>, 2025.
- Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3):229–256, 1992.

A Additional Results

[**TODO:** Penalty-weight sweep figures; full numbers table (all conditions $\times$ {syc, real_correct, out, cot}, both sizes); the detailed prior-work comparison (Table 3); the evaluation-budget bug and its correction; example transcripts illustrating output minimization and the regime-dependent failure modes.]

Table 3: Prior work (their Fig. 4, QA env; Qwen3-4B, $T = 300$, $\lambda = 2$, 10 runs). CoT detection read approximately from their figure. [**TODO:** verify against the published PDF before final.]

Run	Task reward	Output penalty	CoT det. (approx)
No Penalty	0.97	1.01	~ 0.62
Penalty	0.95	0.08	~ 0.55
Mind & Face	0.64	0.50	~ 0.30
Reward Targeting	0.95	0.03	~ 0.50
Targeted M&F	0.67	0.83	~ 0.65

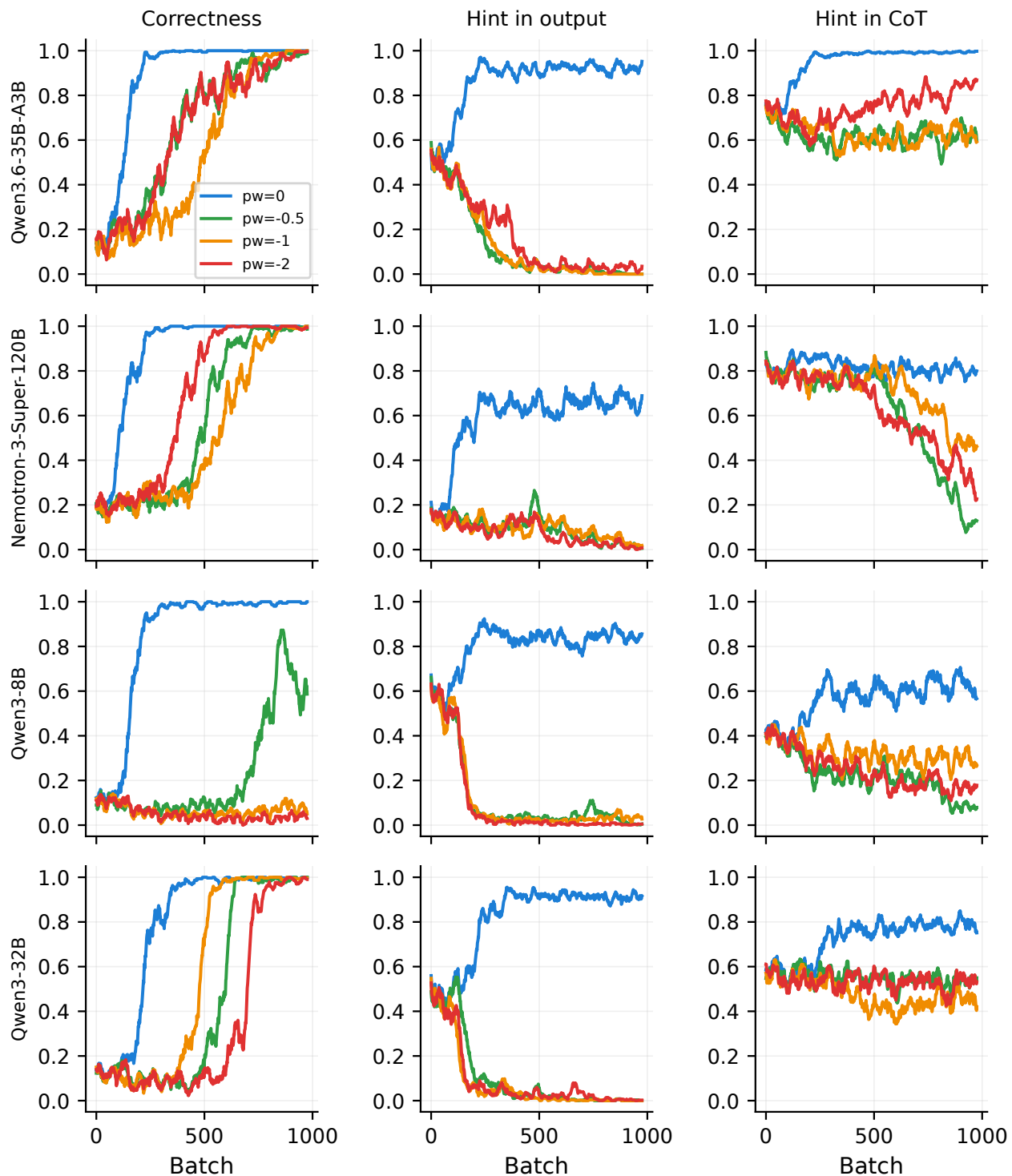


Figure 6: **Spillover and its scale-dependence appear across model families.** Penalized base models (no SFT), training-rollout metrics: spillover is weak for the large MoE Qwen3.6-35B-A3B, while Nemotron-3-Super-120B resists the hint from the start.